

ESPECIFICAÇÃO DO PROJETO DA DISCIPLINA

SISTEMA ONLINE DE GESTÃO E RESERVA DE SALAS

 Integrantes: Carolina Costa, Eduarda Mendonça e Eduarda Moura

PROBLEMA

Atualmente, a gestão de reservas de salas em instituições de ensino, empresas ou espaços públicos é feita, muitas vezes, de maneira manual ou com sistemas pouco eficientes. Isso gera problemas como conflito de agendamentos, falta de controle sobre o uso dos espaços, dificuldades na visualização de disponibilidade e perda de tempo tanto para os solicitantes quanto para os administradores.

SOLUÇÃO PROPOSTA - OPORTUNIDADES

Desenvolver um sistema online que permita a reserva de salas de forma prática, rápida e eficiente. O sistema oferecerá uma interface onde usuários podem consultar a disponibilidade das salas, características (capacidade, recursos como projetor, ar-condicionado etc.) e efetuar reservas, tudo de forma organizada, evitando conflitos e otimizando o uso dos espaços.

STAKEHOLDERS E SUAS NECESSIDADES

- Instituições de ensino (universidades, escolas, centros técnicos): Precisam de um sistema que organize o uso de salas e laboratórios, evitando conflitos de horário, permitindo visualizar disponibilidades e registrar reservas de forma centralizada para apoio à gestão acadêmica.
- Empresas (para reserva de salas de reunião, auditórios, coworkings): Necessitam de uma solução rápida e prática para reservar salas de reunião ou coworking, com informações claras sobre capacidade e recursos, reduzindo conflitos e melhorando a produtividade interna.
- Órgãos públicos ou espaços comunitários: Precisam de uma plataforma simples e transparente para gerenciar o uso de espaços coletivos, permitindo reservas por diferentes setores ou cidadãos e garantindo organização, controle e prestação de contas.

FUNCIONALIDADES

- Login e autenticação de usuários (alunos, professores, funcionários, administradores)
- Cadastro de salas com descrição, capacidade e recursos disponíveis
- Visualização de calendário com disponibilidade em tempo real
- Solicitação e confirmação de reserva de salas
- Prevenção de conflitos de horários automaticamente
- Cancelamento e alteração de reservas
- Notificações por e-mail ou app sobre confirmações, lembretes e alterações
- Relatórios administrativos de uso das salas (taxa de ocupação, horários mais demandados, etc.)

REQUISITOS NÃO-FUNCIONAIS

RNF001 – Usabilidade

O sistema deve ser fácil de usar e intuitivo para todos os perfis de usuários.

RNF002 – Desempenho

As informações devem carregar em até 3 segundos para garantir rapidez nas operações.

RNF003 – Confiabilidade

O sistema deve processar dados corretamente, evitando falhas ou perdas de informação.

RNF004 – Disponibilidade

O sistema deve estar acessível pelo menos 99% do tempo.

RNF005 – Segurança

O acesso e os dados dos usuários devem ser protegidos por autenticação e permissões adequadas.

RNF006 – Integridade de Dados

As informações devem permanecer consistentes, sem duplicações ou conflitos.

RNF007 – Compatibilidade

O sistema deve funcionar bem em diversos dispositivos e navegadores.

RNF008 – Escalabilidade

O sistema deve suportar aumento de usuários e reservas sem perda de desempenho.

RNF009 – Manutenibilidade

A estrutura deve facilitar atualizações, correções e melhorias.

RNF010 – Acessibilidade

O sistema deve ser acessível a usuários com diferentes necessidades.

RNF011 – Privacidade

Os dados pessoais devem ser tratados conforme a LGPD.

REQUISITOS FUNCIONAIS

RF001 – Login
RF002 – Cadastrar Sala
RF003 – Exibir Disponibilidade
RF004 – Solicitar Reserva
RF005 – Verificar Conflitos
RF006 – Alterar Reserva
RF007 – Cancelar Reserva
RF008 – Relatórios
RF009 – Designar Avaliadores
RF010 – Avaliar Sala
RF011 – Gerenciar Reservas
RF012 – Cadastrar Espaço
RF013 – Editar Espaço
RF014 – Excluir Espaço
RF015 – Listar Espaços
RF016 – Visualizar Detalhes do Espaço

CASOS DE USO

CDU001 - Realizar login
Administrador, avaliador ou cliente inserem credenciais para acessar o sistema.

CDU002 - Gerenciar sala
Administrador visualiza, altera ou exclui dados da sala (capacidade, recursos etc.).

CDU003 - Gerar relatórios
Administrador gera relatórios administrativos sobre o uso das salas.

CDU004 - Adicionar avaliador
Administrador define quem pode aprovar ou recusar solicitações de reserva.

CDU005 - Avaliar solicitação de reserva
Avaliador aprova ou recusa uma solicitação feita por um usuário.

CDU006 - Gerenciar reserva
Usuário visualiza, altera ou exclui reservas feitas.

CDU007 - Visualizar disponibilidade
Usuário consulta o calendário de horários e salas disponíveis.

CDU008- Avaliar reserva
Após utilizar a sala, o usuário avalia sua experiência.

DIAGRAMA DE CDU

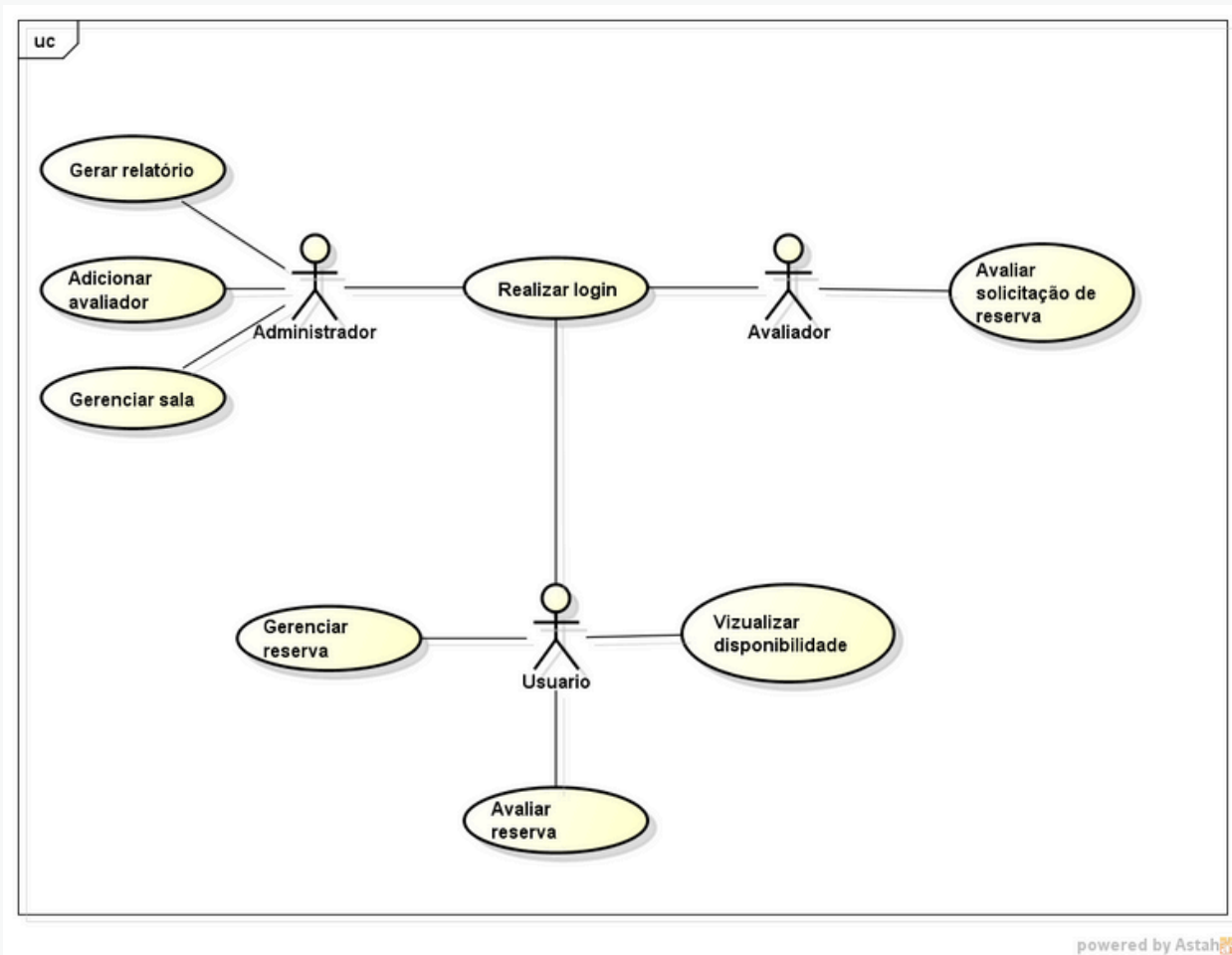


DIAGRAMA ER



DIAGRAMA ER (DRAW.IO)

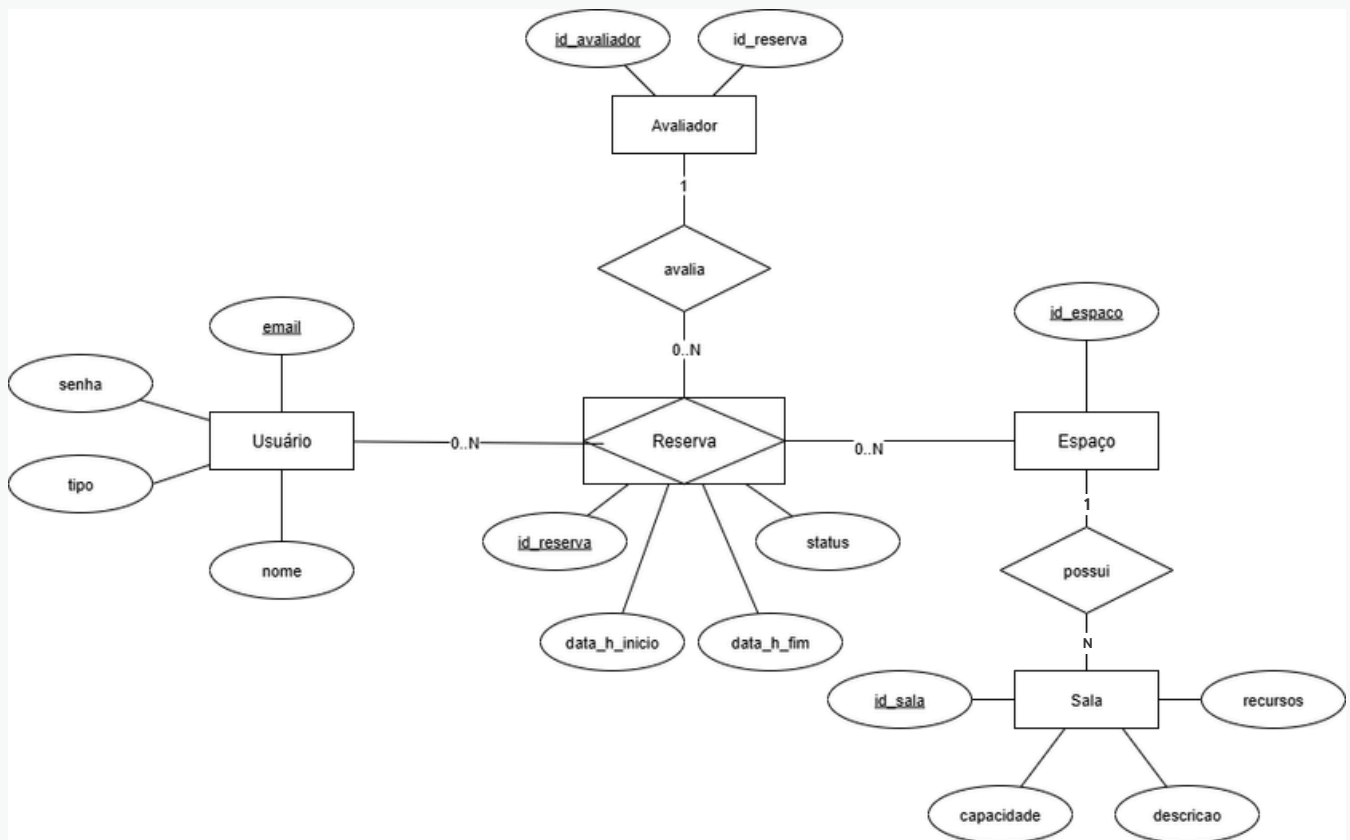
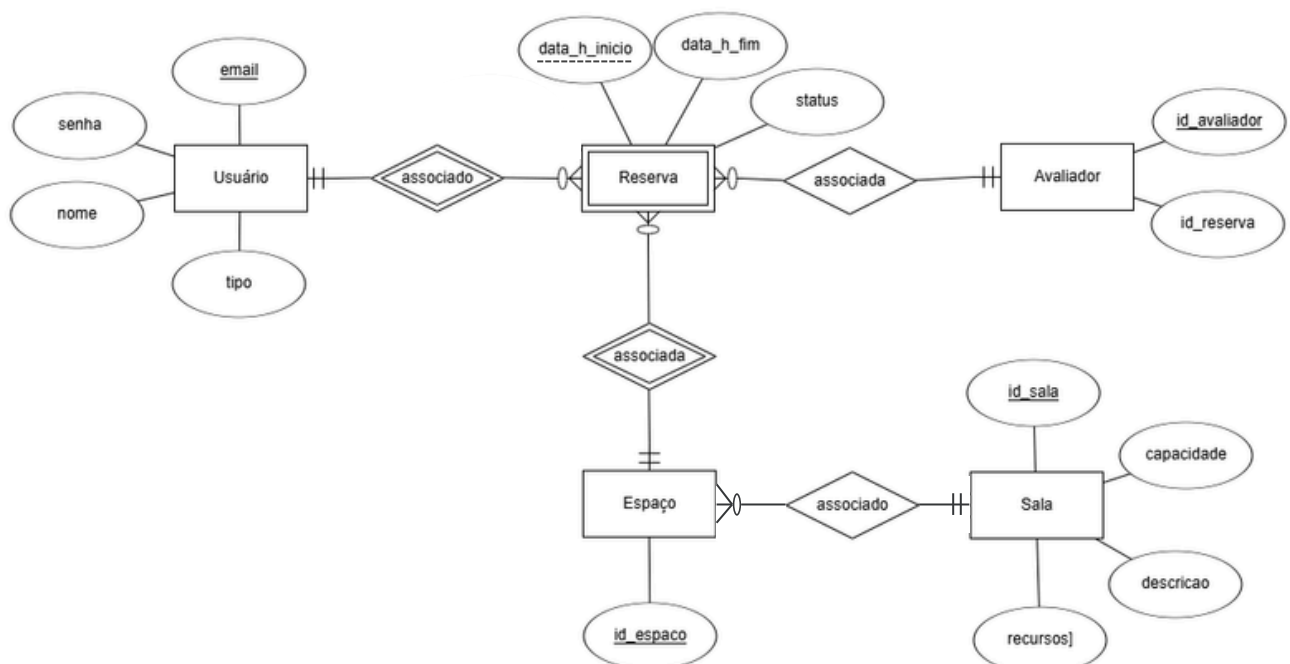
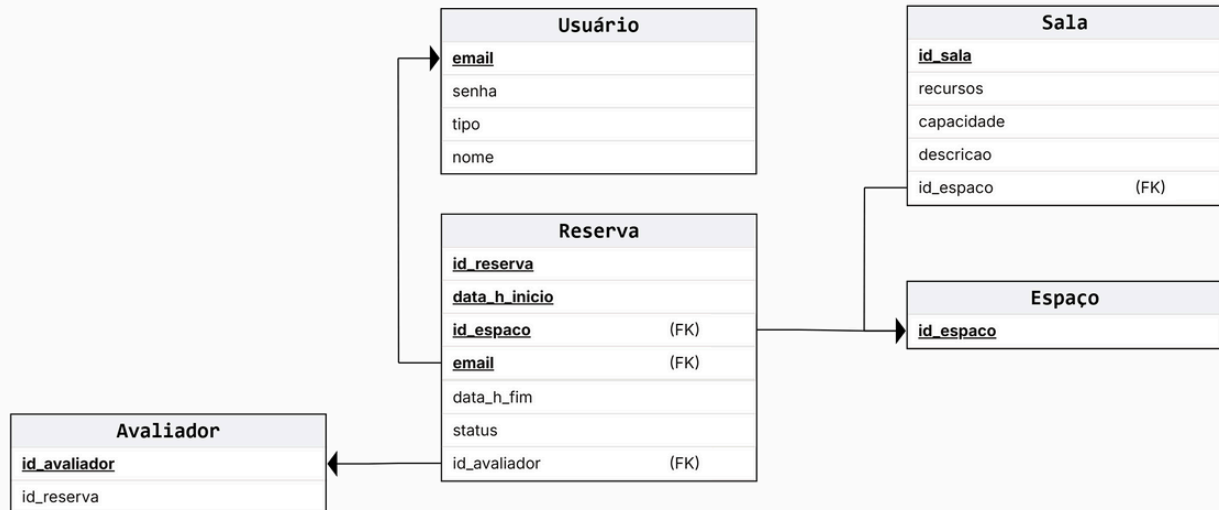


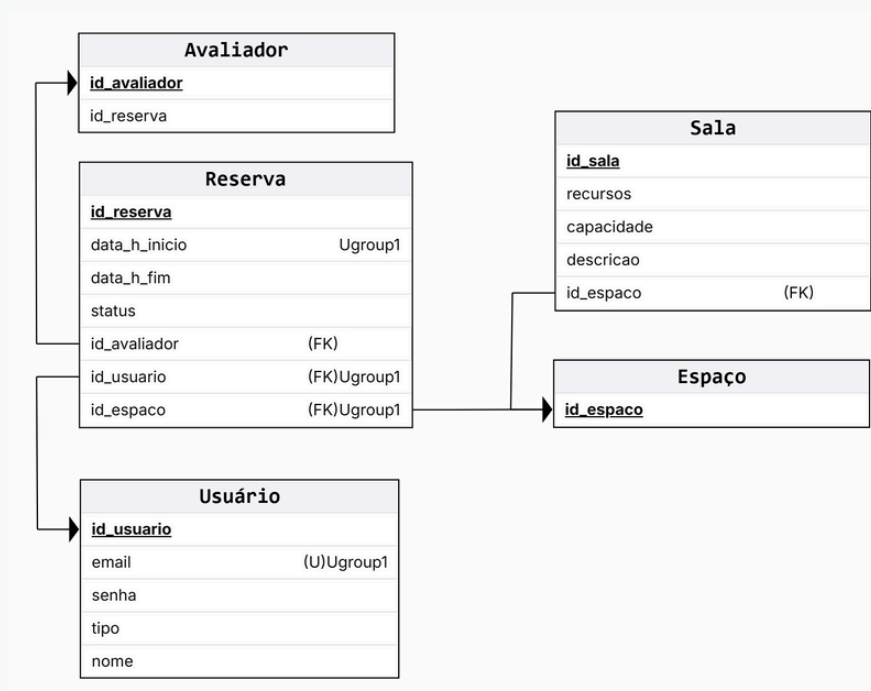
DIAGRAMA ER INTERMEDIÁRIO (ERDPLUS)



MODELO RELACIONAL



MODELO RELACIONAL SEM AS PKS COMPOSTAS



DICIONÁRIO DE DADOS DO MODELO RELACIONAL

Usuário					
Nome do Campo	Tipo	Chave	Obrigatório	Descrição	Restrição
email	VARCHAR	PK	Sim	Identificador único do usuário	Unique Group
senha	VARCHAR	-	Sim	Senha do usuário para acesso ao sistema	-
tipo	VARCHAR	-	Sim	Define o tipo de usuário	-
nome	VARCHAR	-	Sim	Nome completo do usuário	-
id_usuario	INT	PK	Sim	Identificador único do usuário	-

Reserva					
Nome do Campo	Tipo	Chave	Obrigatório	Descrição	Restrições
id_reserva	INT	PK	Sim	Identificador único de reserva	-
id_espaco	INT	FK	Sim	Identifica a sala reservada	Unique Group
dt_h_inicio	DATETIME	-	Sim	Data e hora de início da reserva	Unique Group
dt_h_fim	DATETIME	-	Sim	Data de hora do término da reserva	-
status	VARCHAR	-	Sim	Situação atual da reserva ("pendente", "aprovada", "rejeitada")	-
id_avaliador	INT	FK	Sim	Identifica o avaliador responsável pela análise da reserva	-
id_usuario	INT	FK	Sim	Identificador único do usuário	-

Avaliador				
Nome do Campo	Tipo	Chave	Obrigatório	Descrição
id_avaliador	INT	PK	Sim	Identificador único do avaliador
id_reserva	INT	-	Sim	Identifica a reserva associada ao avaliador

Sala				
Nome do Campo	Tipo	Chave	Obrigatório	Descrição
id_sala	INT	PK	Sim	Identificador único do usuário
recursos	VARCHAR	-	Sim	Identifica o espaço pertencente à sala
capacidade	INT	-	Sim	Lista de recursos disponíveis
descricao	VARCHAR	-	Sim	Capacidade máxima de pessoas
id_espaco	INT	FK	Sim	Descrição adicional sobre a sala

Espaço				
Nome do Campo	Tipo	Chave	Obrigatório	Descrição
id_espaco	INT	PK	Sim	Identificador único do espaço

CÓDIGO SQL - COMANDOS DDL

-- Criação da tabela Espaço

```
CREATE TABLE Espaco (  
    id_espaco INT PRIMARY KEY  
);
```

-- Criação da tabela Usuário

```
CREATE TABLE Usuario (  
    id_usuario INT PRIMARY KEY,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    senha VARCHAR(100) NOT NULL,  
    tipo VARCHAR(50) NOT NULL,  
    nome VARCHAR(150) NOT NULL  
);
```

-- Criação da tabela Sala

```
CREATE TABLE Sala (  
    id_sala INT PRIMARY KEY,  
    recursos VARCHAR(255) NOT NULL,  
    capacidade INT NOT NULL,  
    descricao VARCHAR(255) NOT NULL,  
    id_espaco INT NOT NULL,  
    FOREIGN KEY (id_espaco) REFERENCES Espaco(id_espaco)  
);
```

-- Criação da tabela Avaliador

```
CREATE TABLE Avaliador (  
    id_avaliador INT PRIMARY KEY  
);
```

-- Criação da tabela Reserva

```
CREATE TABLE Reserva (  
    id_reserva INT PRIMARY KEY,  
    id_espaco INT NOT NULL,  
    dt_h_inicio DATETIME NOT NULL,  
    dt_h_fim DATETIME NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    id_avaliador INT NOT NULL,  
    id_usuario INT NOT NULL,  
    FOREIGN KEY (id_espaco) REFERENCES Espaco(id_espaco),  
    FOREIGN KEY (id_avaliador) REFERENCES Avaliador(id_avaliador),  
    FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario),  
    UNIQUE (id_espaco, dt_h_inicio),  
    UNIQUE (id_espaco, dt_h_fim)  
);
```

CÓDIGO SQL - COMANDOS INSERTS

-- Inserção de dados na tabela Espaco

```
INSERT INTO Espaco (id_espaco) VALUES (101);
```

```
INSERT INTO Espaco (id_espaco) VALUES (102);
```

```
INSERT INTO Espaco (id_espaco) VALUES (103);
```

```
INSERT INTO Espaco (id_espaco) VALUES (104);
```

```
INSERT INTO Espaco (id_espaco) VALUES (105);
```

-- Inserção de dados na tabela Usuario

```
INSERT INTO Usuario (id_usuario, email, senha, tipo, nome) VALUES (1, 'ana@escola.edu', '12345', 'Professor', 'Ana Silva');
```

```
INSERT INTO Usuario (id_usuario, email, senha, tipo, nome) VALUES (2, 'carlos@escola.edu', '12345', 'Aluno', 'Carlos Souza');
```

```
INSERT INTO Usuario (id_usuario, email, senha, tipo, nome) VALUES (3, 'admin@escola.edu', 'admin123', 'Administrador', 'Roberto Admin');
```

```
INSERT INTO Usuario (id_usuario, email, senha, tipo, nome) VALUES (4, 'julia@escola.edu', '12345', 'Aluno', 'Julia Mendes');
```

```
INSERT INTO Usuario (id_usuario, email, senha, tipo, nome) VALUES (5, 'coord@escola.edu', '54321', 'Coordenador', 'Mariana Coordenadora');
```

-- Inserção de dados na tabela Avaliador

```
INSERT INTO Avaliador (id_avalizador) VALUES (10);
```

```
INSERT INTO Avaliador (id_avalizador) VALUES (20);
```

```
INSERT INTO Avaliador (id_avalizador) VALUES (30);
```

```
INSERT INTO Avaliador (id_avalizador) VALUES (40);
```

```
INSERT INTO Avaliador (id_avalizador) VALUES (50);
```

-- Inserção de dados na tabela Sala

```
INSERT INTO Sala (id_sala, recursos, capacidade, descricao, id_espaco) VALUES (1, 'Projektor, Ar Condicionado', 30, 'Sala de Aula Teórica', 101);
```

```
INSERT INTO Sala (id_sala, recursos, capacidade, descricao, id_espaco) VALUES (2, 'Computadores, Lousa Digital', 20, 'Laboratório de Informática', 102);
```

```
INSERT INTO Sala (id_sala, recursos, capacidade, descricao, id_espaco) VALUES (3, 'Mesa de Reunião, TV', 10, 'Sala de Reuniões', 103);
```

```
INSERT INTO Sala (id_sala, recursos, capacidade, descricao, id_espaco) VALUES (4, 'Palco, Som', 100, 'Auditório Principal', 104);
```

```
INSERT INTO Sala (id_sala, recursos, capacidade, descricao, id_espaco) VALUES (5, 'Mesas em grupo', 25, 'Sala de Estudos', 105);
```

CÓDIGO SQL - COMANDOS INSERT

-- Inserção de dados na tabela Reserva

```
INSERT INTO Reserva (id_reserva, id_espaco, dt_h_inicio, dt_h_fim, status,  
id_avalizador, id_usuario) VALUES (1001, 101, '2023-10-10 08:00:00', '2023-10-10  
10:00:00', 'Aprovada', 10, 1);
```

```
INSERT INTO Reserva (id_reserva, id_espaco, dt_h_inicio, dt_h_fim, status,  
id_avalizador, id_usuario) VALUES (1002, 102, '2023-10-11 14:00:00', '2023-10-11  
16:00:00', 'Pendente', 20, 2);
```

```
INSERT INTO Reserva (id_reserva, id_espaco, dt_h_inicio, dt_h_fim, status,  
id_avalizador, id_usuario) VALUES (1003, 103, '2023-10-12 09:00:00', '2023-10-12  
11:00:00', 'Rejeitada', 10, 4);
```

```
INSERT INTO Reserva (id_reserva, id_espaco, dt_h_inicio, dt_h_fim, status,  
id_avalizador, id_usuario) VALUES (1004, 104, '2023-10-13 19:00:00', '2023-10-13  
22:00:00', 'Aprovada', 30, 5);
```

```
INSERT INTO Reserva (id_reserva, id_espaco, dt_h_inicio, dt_h_fim, status,  
id_avalizador, id_usuario) VALUES (1005, 101, '2023-10-14 08:00:00', '2023-10-14  
10:00:00', 'Pendente', 10, 1);
```

CÓDIGO SQL - COMANDOS DML*

-- (a) Um comando UPDATE para atualizar um campo de todos os registros de uma tabela.-- Atualiza o status de todas as reservas para 'Em Análise' (reset global).

```
UPDATE Reserva SET status = 'Em Análise';
```

-- (b) Um comando UPDATE para atualizar um campo dos registros que satisfazem uma condição simples.-- Atualiza a capacidade da Sala 1.

```
UPDATE Sala SET capacidade = 35 WHERE id_sala = 1;
```

-- (c) Um comando UPDATE para atualizar um campo dos registros que satisfazem uma condição composta.-- Aprova reservas pendentes feitas pelo usuário 1.

```
UPDATE Reserva SET status = 'Aprovada' WHERE status = 'Pendente' AND id_usuario = 1;
```

-- (d) Um comando UPDATE para atualizar dois campos dos registros que satisfazem uma condição.-- Atualiza nome e email de um usuário específico.

```
UPDATE Usuario SET nome = 'Carlos Silva', email = 'carlos.silva@escola.edu' WHERE id_usuario = 2;
```

-- (e) Um comando UPDATE para atualizar um campo usando o antigo valor desse campo.-- Aumenta a capacidade de todas as salas em 5 lugares.

```
UPDATE Sala SET capacidade = capacidade + 5;
```

-- (f) Um comando UPDATE para atualizar um campo usando uma função pré-definida.-- Coloca o nome de todos os usuários em letras maiúsculas.

```
UPDATE Usuario SET nome = UPPER(nome);
```

-- (g) Um comando DELETE para remover todos os registros de uma tabela.--

Remove todas as reservas (Cuidado: limpa a tabela).DELETE FROM Reserva;

-- (h) Um comando DELETE para remover os registros que satisfazem uma condição simples.-- Remove salas com capacidade menor que 10.DELETE FROM Sala WHERE

capacidade < 10;

-- (i) Um comando DELETE para remover os registros que satisfazer uma condição composta.-- Remove reservas rejeitadas de um usuário específico.DELETE FROM

Reserva WHERE status = 'Rejeitada' AND id_usuario = 4;

-- (j) Um comando DELETE para remover os registros usando uma função agregada na condição.-- Remove salas que têm capacidade abaixo da média de todas as

salas.DELETE FROM Sala WHERE capacidade < (SELECT AVG(capacidade) FROM Sala);

CÓDIGO SQL - COMANDOS SELECT*

-- (a) Um comando SELECT para listar todos os campos de todos os registros de uma tabela.

-- Lista todos os usuários cadastrados.

```
SELECT * FROM Usuario;
```

-- (b) Um comando SELECT para listar alguns campos dos registros que satisfazem uma condição simples.

-- Lista nome e email apenas dos usuários do tipo 'Professor'.

```
SELECT nome, email FROM Usuario WHERE tipo = 'Professor';
```

-- (c) Um comando SELECT para listar alguns campos dos registros que satisfazer uma condição composta.

-- Lista reservas que estão 'Aprovada' e ocorrem na sala/espço 101.

```
SELECT id_reserva, dt_h_inicio FROM Reserva WHERE status = 'Aprovada' AND id_espaco = 101;
```

-- (d) Um comando SELECT usando GROUP BY com os campos dos registros que satisfazem uma condição.

-- Conta quantas reservas existem por status (aprovada, pendente, etc.).

```
SELECT status, COUNT(*) as total_reservas FROM Reserva GROUP BY status;
```

-- (e) Um comando SELECT contendo como condição outro SELECT (SELECTs aninhados).

-- Lista os nomes dos usuários que fizeram reservas no Espaço 101.

```
SELECT nome FROM Usuario WHERE id_usuario IN (SELECT id_usuario FROM Reserva WHERE id_espaco = 101);
```

CÓDIGO SQL - COMANDOS JUNCOES*

-- (a) Três comandos SELECT com junção interna (INNER JOIN).

-- 1. Lista detalhes da reserva junto com o nome do usuário que a fez.

```
SELECT R.id_reserva, R.dt_h_inicio, U.nome  
FROM Reserva R  
INNER JOIN Usuario U ON R.id_usuario = U.id_usuario;
```

-- 2. Lista a descrição da sala associada a cada reserva.

-- (Join entre Reserva, Espaço e Sala, pois Reserva liga com Espaço, e Sala liga com Espaço)

```
SELECT R.id_reserva, S.descricao, S.capacidade  
FROM Reserva R  
INNER JOIN Sala S ON R.id_espaco = S.id_espaco;
```

-- 3. Lista as reservas e o ID do avaliador responsável.

```
SELECT R.id_reserva, R.status, A.id_avaliador  
FROM Reserva R  
INNER JOIN Avaliador A ON R.id_avaliador = A.id_avaliador;
```

-- (b) Dois comandos SELECT com junção externa parcial (LEFT ou RIGHT OUTER JOIN).

-- 1. LEFT JOIN: Lista todos os usuários e suas reservas (mesmo quem não tem reserva aparece).

```
SELECT U.nome, R.id_reserva, R.status  
FROM Usuario U  
LEFT JOIN Reserva R ON U.id_usuario = R.id_usuario;
```

-- 2. RIGHT JOIN: Lista todos os Espaços e suas Salas (para ver se há espaços cadastrados sem sala definida).

```
SELECT E.id_espaco, S.descricao  
FROM Sala S  
RIGHT JOIN Espaco E ON S.id_espaco = E.id_espaco;
```

-- (c) Um comando SELECT com junção externa total (FULL OUTER JOIN).

-- Lista todas as reservas e todos os usuários, combinando onde possível e trazendo NULL onde não há correspondência.

-- (Nota: MySQL não suporta FULL OUTER JOIN nativo, mas PostgreSQL sim).

```
SELECT U.nome, R.id_reserva  
FROM Usuario U  
FULL OUTER JOIN Reserva R ON U.id_usuario = R.id_usuario;
```